МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет “Львівська політехніка”
Інститут комп'ютерних наук та інформаційних технологій
Кафедра систем автоматизованого проектування



Звіт

до лабораторної роботи № 5

з курсу

«Організація баз даних та знань»

на тему: “Написання запитів SELECT. Ч. 2”

Виконав:
студент гр. ПП-24
Лемішка А.А.

Прийняла:
Спас Н.Я.

ЛЬВІВ – 2026

Мета роботи: набути практичних навичок написання складних запитів SELECT у СУБД MySQL для бази даних marketplace_3d: створення корелюючих та некорелюючих підзапитів, застосування різних видів з'єднання таблиць (INNER JOIN, LEFT/RIGHT/CROSS JOIN), використання умовного оператора CASE, теоретико-множинних операцій (UNION, UNION ALL, INTERSECT, EXCEPT), а також горизонтального секціонування таблиць (RANGE, LIST, HASH) у середовищі MySQL Workbench.

Хід роботи

Завдання 1. Написання різних типів підзапитів (корелюючі та некорелюючі)

Запит 1.1. Скалярний некорелюючий підзапит у SELECT (різниця цін моделі та середньої ціни каталогу):

```
SELECT name, price, (SELECT AVG(price) FROM model_3d) AS avg_price FROM model_3d LIMIT 5;
```

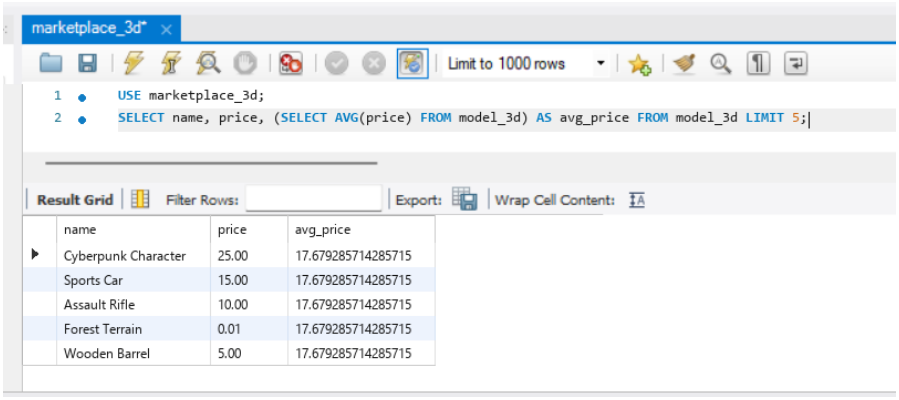


Рис. 1.1. Результат виконання запиту 1.1

Запит 1.2. Некорелюючий підзапит у WHERE з оператором порівняння (моделі дорожчі за середню ціну):

```
SELECT name, price FROM model_3d WHERE price > (SELECT AVG(price) FROM model_3d) LIMIT 5;
```

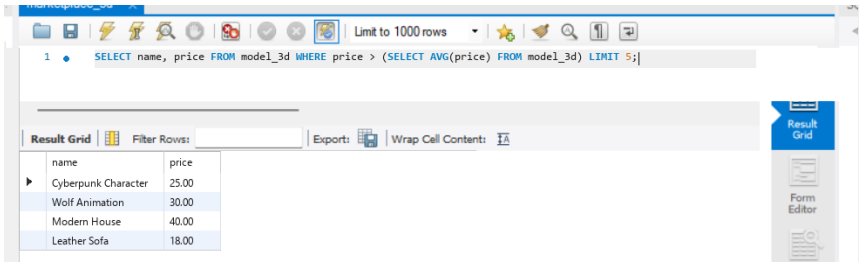


Рис. 1.2. Результат виконання запиту 1.2

Запит 1.3. Некорелюючий підзапит з оператором IN (моделі авторів з високим рейтингом >= 4.70):

```
SELECT name, price FROM model_3d WHERE id_seller IN (SELECT id_seller FROM seller WHERE
rating >= 4.70) LIMIT 5;
```

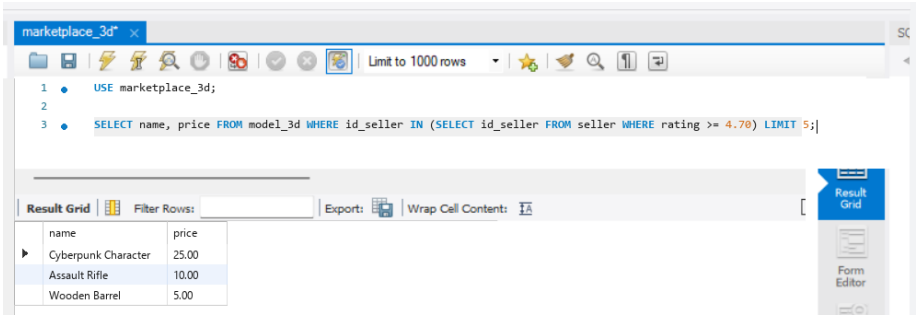


Рис. 1.3. Результат виконання запиту 1.3

Запит 1.4. Корелюючий підзапит з оператором EXISTS (покупці, що мають хоча б одне замовлення):

```
SELECT email FROM buyer b WHERE EXISTS (SELECT 1 FROM cart c WHERE c.id_buyer = b.id_buyer)
LIMIT 5;
```

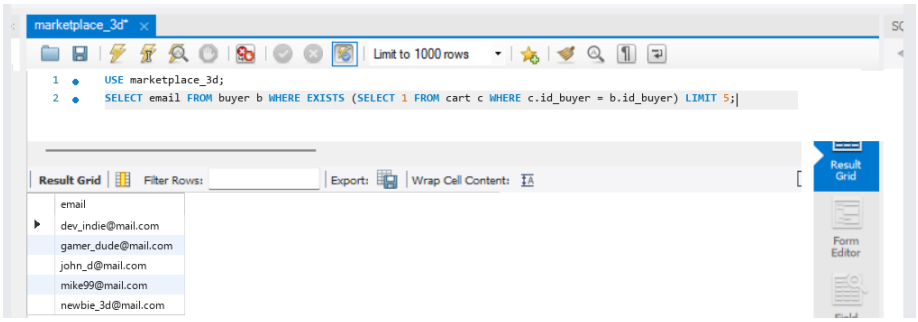


Рис. 1.4. Результат виконання запиту 1.4

Запит 1.5. Підзапит у виразі обчислення цін для автора з найвищим рейтингом:

```
SELECT name, price, ROUND(price * 1.10, 2) AS new_price FROM model_3d WHERE id_seller =
(SELECT id_seller FROM seller ORDER BY rating DESC LIMIT 1) LIMIT 5;
```

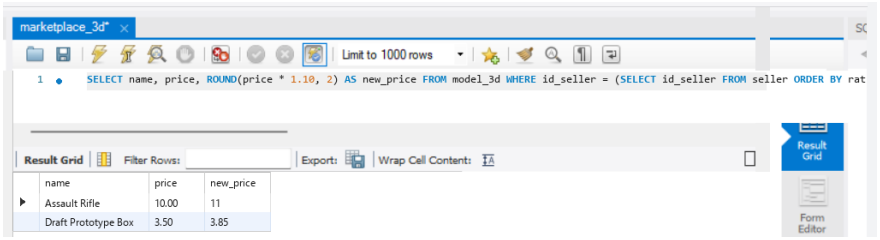


Рис. 1.5. Результат виконання запиту 1.5

Завдання 2. Формування зведених таблиць з використанням операторів INNER JOIN та OUTER JOIN

Запит 2.1. Внутрішнє з'єднання двох таблиць (INNER JOIN моделей та категорій):

```
SELECT m.name, c.name AS category FROM model_3d m JOIN category c ON m.id_category =
c.id_category LIMIT 5;
```

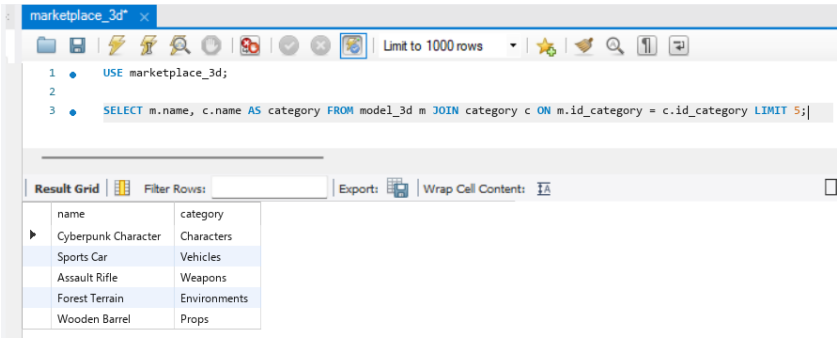


Рис. 2.1. Результат виконання запиту 2.1

Запит 2.2. Внутрішнє з'єднання чотирьох таблиць (модель, автор, категорія, ліцензія):

```
SELECT m.name, s.nickname, c.name AS category, l.license_name FROM model_3d m JOIN seller s
ON m.id_seller = s.id_seller JOIN category c ON m.id_category = c.id_category JOIN license
l ON m.id_license = l.id_license LIMIT 5;
```

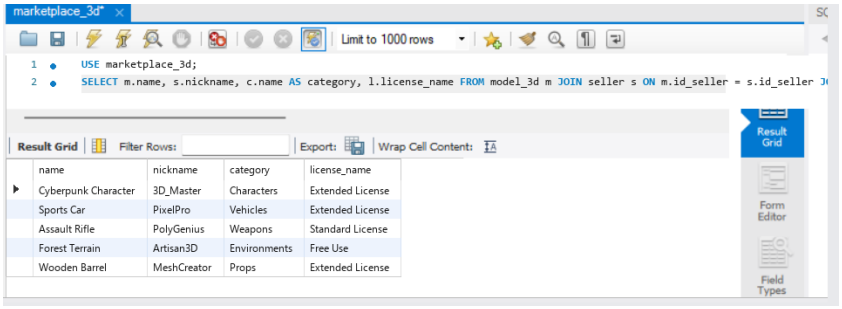


Рис. 2.2. Результат виконання запиту 2.2

Запит 2.3. Ліве зовнішнє з'єднання (LEFT JOIN авторів без створених моделей з перевіркою IS NULL):

```
SELECT s.nickname, m.name AS model_name FROM seller s LEFT JOIN model_3d m ON s.id_seller =
m.id_seller WHERE m.id_model IS NULL;
```

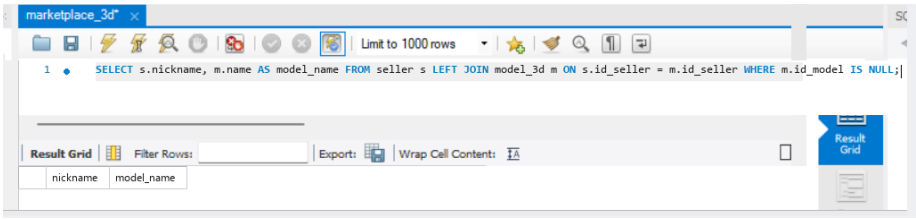


Рис. 2.3. Результат виконання запиту 2.3

Запит 2.4. Праве зовнішнє з'єднання (RIGHT JOIN замовлень та зареєстрованих покупців):

```
SELECT b.email, c.id_cart, c.total_amount FROM buyer b RIGHT JOIN cart c ON b.id_buyer =
c.id_buyer LIMIT 5;
```

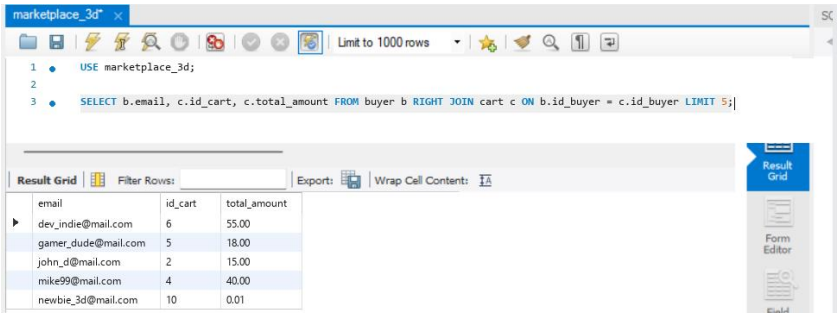


Рис. 2.4. Результат виконання запиту 2.4

Запит 2.5. Декартовий добуток (CROSS JOIN категорій та ліцензій):

```
SELECT c.name AS category, l.license_name AS license FROM category c CROSS JOIN license l
LIMIT 5;
```

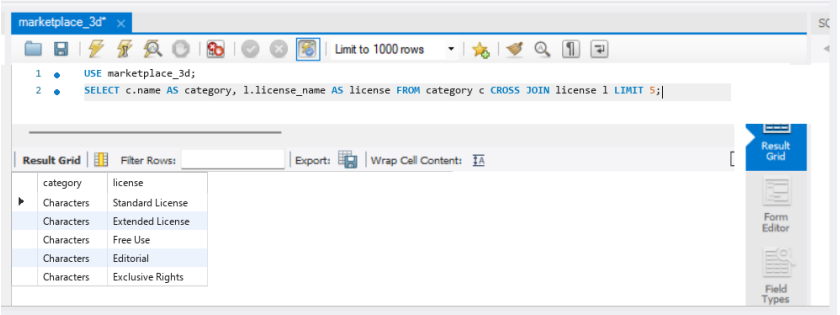


Рис. 2.5. Результат виконання запиту 2.5

Завдання 3. Створення запитів з використанням функції CASE (з агрегатними функціями та без них)

Запит 3.1. Простий оператор CASE без агрегації (перетворення ознаки наявності текстур):

```
SELECT name, has_texture,
    CASE has_texture
        WHEN 'Yes' THEN 'Текстурована'
        ELSE 'Базова'
    END AS texture_status
FROM model_3d
LIMIT 5;
```

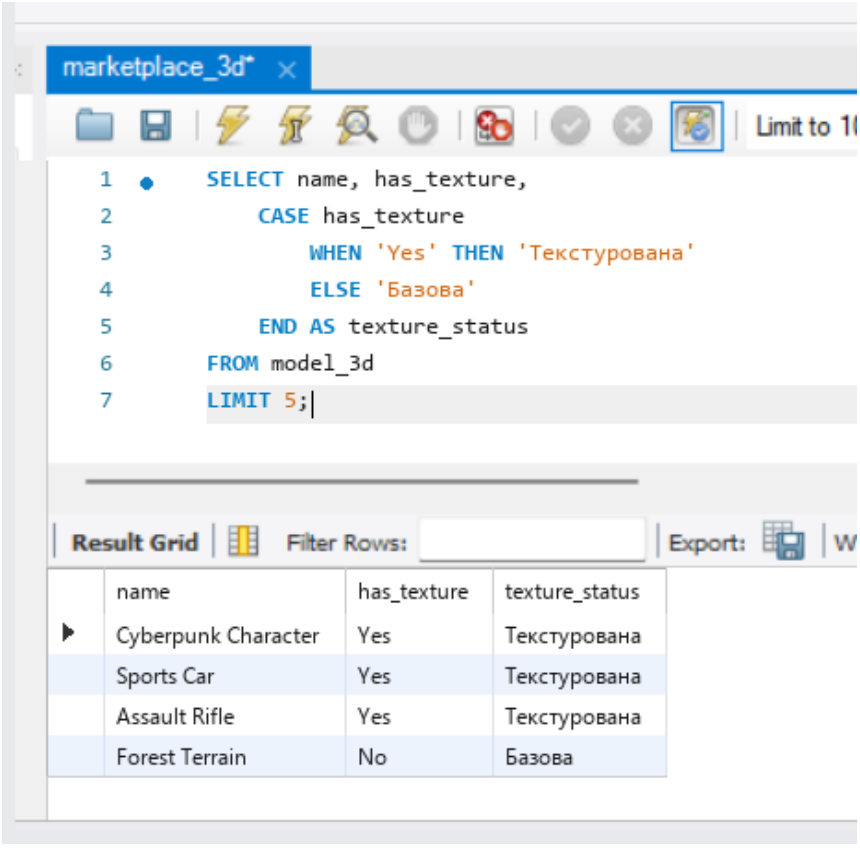


Рис. 3.1. Результат виконання запиту 3.1

Запит 3.2. Пошуковий оператор CASE (класифікація моделей за кількістю полігонів Low/Mid/High-Poly):

```
SELECT name, polygon_count,
    CASE
        WHEN polygon_count < 20000
            THEN 'Low-Poly'
        WHEN polygon_count <= 80000
            THEN 'Mid-Poly'
        ELSE 'High-Poly'
    END AS poly_grade
FROM model_3d
LIMIT 5;
```

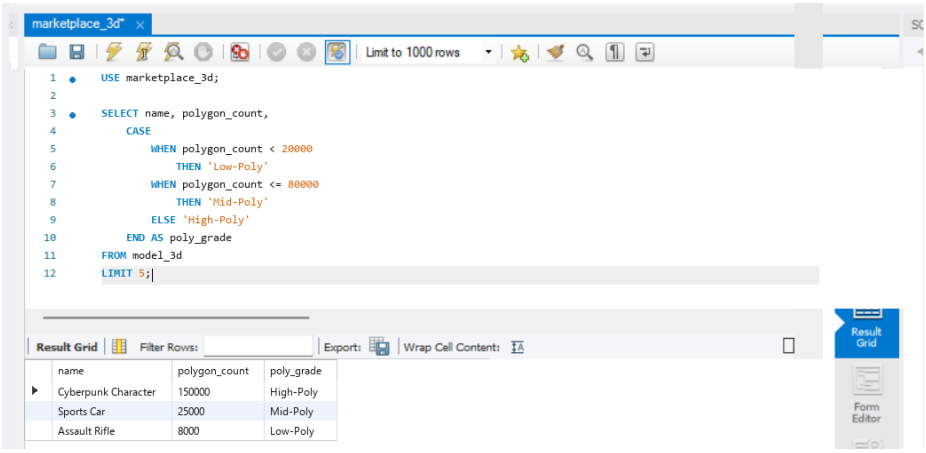


Рис. 3.2. Результат виконання запиту 3.2

Запит 3.3. Використання CASE в агрегатній функції SUM (підрахунок кількості замовлень за статусами):

```
SELECT id_buyer,
    SUM(CASE WHEN status = 'Completed' THEN 1 ELSE 0 END) AS done_cnt,
    SUM(CASE WHEN status = 'Active' THEN 1 ELSE 0 END) AS active_cnt
FROM cart
GROUP BY id_buyer
LIMIT 5;
```

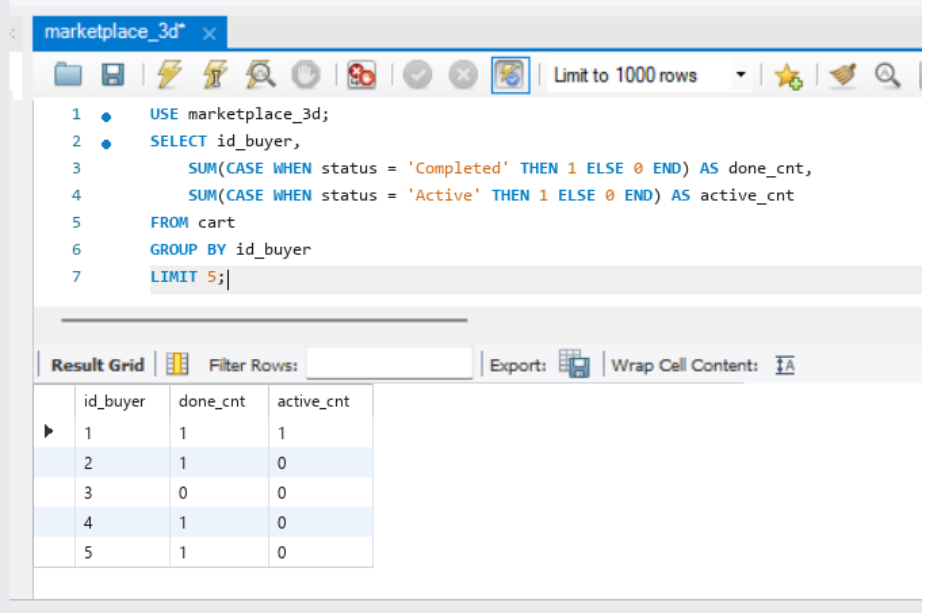


Рис. 3.3. Результат виконання запиту 3.3

Запит 3.4. Умовне обчислення цін зі знижкою 15% для преміум-моделей:

```
SELECT name, price,
    CASE
        WHEN price > 60
            THEN ROUND(price * 0.85, 2)
        ELSE price
    END AS sale_price
FROM model_3d
LIMIT 5;
```

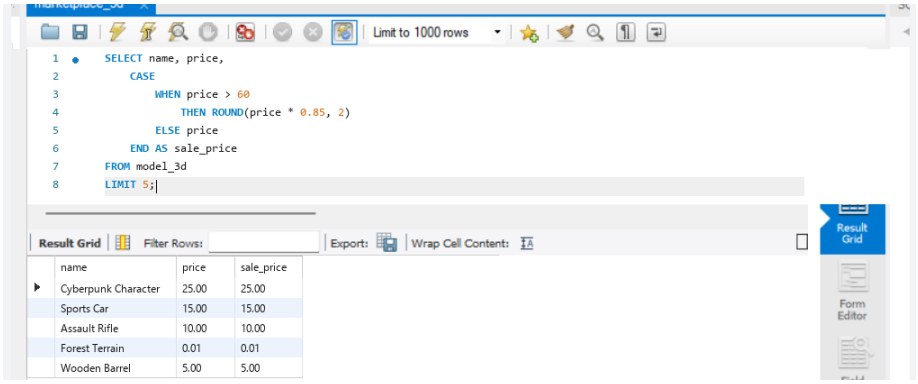


Рис. 3.4. Результат виконання запиту 3.4

Запит 3.5. Класифікація покупців за статусом лояльності та датою реєстрації:

```
SELECT email,
    CASE
        WHEN registration_date < '2024-01-01'
            THEN 'VIP Клієнт'
        ELSE 'Новий'
    END AS client_status
FROM buyer
LIMIT 5;
```

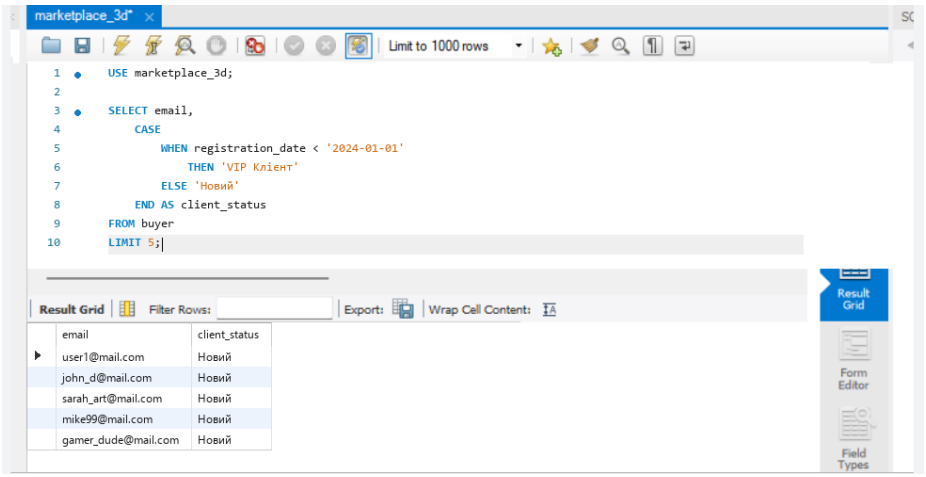


Рис. 3.5. Результат виконання запиту 3.5

Завдання 4. Реалізація теоретико-множинних операцій реляційної алгебри (UNION, INTERSECT, EXCEPT)

Запит 4.1. Об'єднання унікальних значень UNION (контакти користувачів системи):

```
SELECT nickname AS user_contact FROM seller
UNION
SELECT email AS user_contact FROM buyer
LIMIT 5;
```

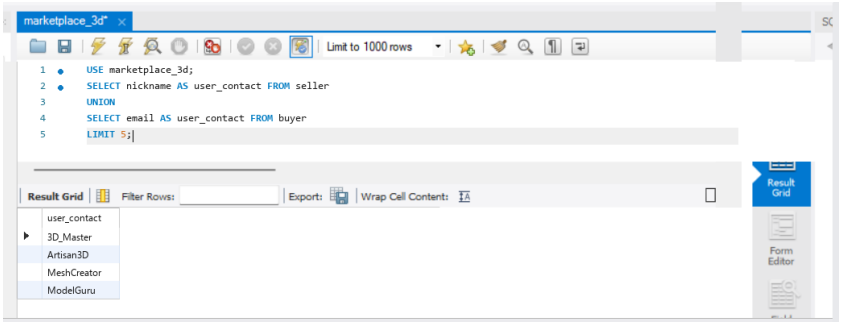


Рис. 4.1. Результат виконання запиту 4.1

Запит 4.2. Об'єднання всіх записів UNION ALL зі збереженням дублікатів:

```
SELECT nickname AS user_contact FROM seller
UNION ALL
SELECT email AS user_contact FROM buyer
LIMIT 5;
```

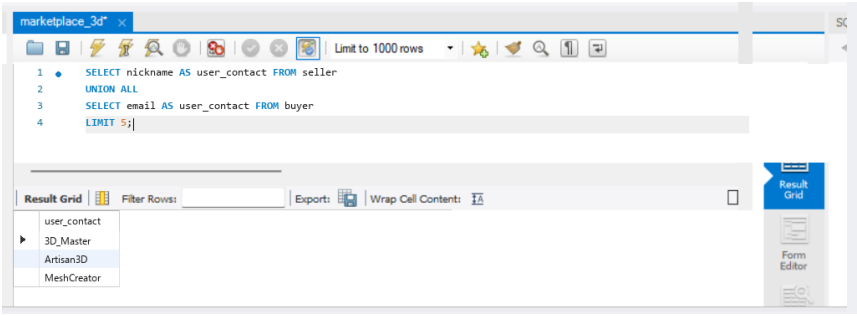


Рис. 4.2. Результат виконання запиту 4.2

Запит 4.3. Перетин множин INTERSECT (покупці, які одночасно залишали відгуки на маркетплейсі):

```
SELECT id_buyer AS common_id FROM cart
INTERSECT
SELECT id_buyer FROM review;
```

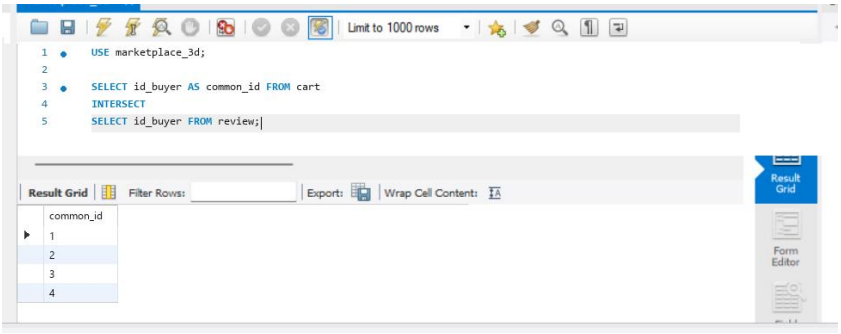


Рис. 4.3. Результат виконання запиту 4.3

Запит 4.4. Різниця множин EXCEPT (покупці, які зареєстровані, але ще не здійснювали замовлень):

```
SELECT id_buyer AS registered_only FROM buyer
EXCEPT
SELECT id_buyer FROM cart;
```

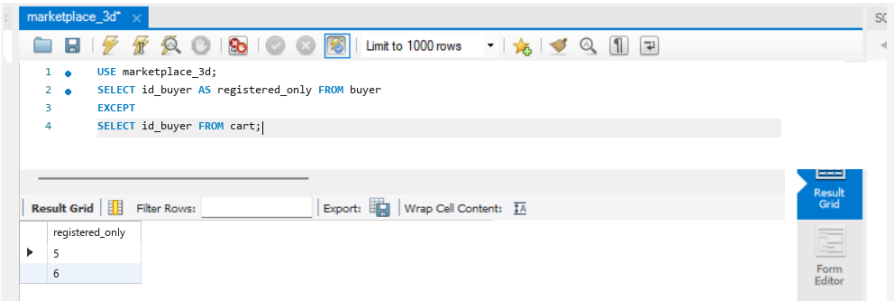


Рис. 4.4. Результат виконання запиту 4.4

Запит 4.5. Комбінована операція UNION з фільтрацією та сортуванням результатів:

```
SELECT name, price FROM model_3d WHERE price > 60
UNION
SELECT name, price FROM model_3d WHERE id_category = 1
ORDER BY price DESC
LIMIT 5;
```

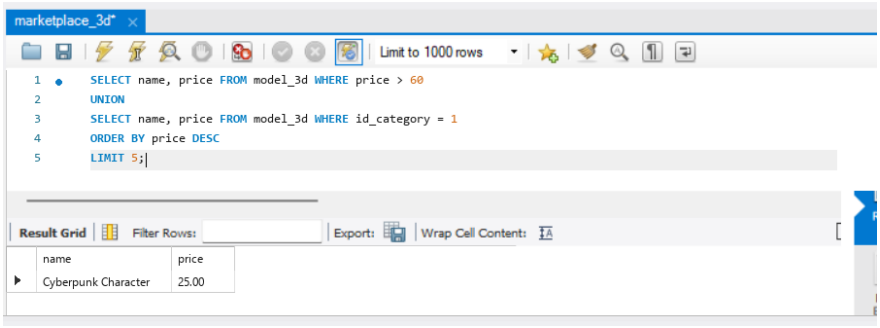


Рис. 4.5. Результат виконання запиту 4.5

Завдання 5. Розділення таблиць бази даних за допомогою RANGE, LIST і HASH

Запит 5.1. Розділення таблиці cart за допомогою RANGE (секціонування замовлень за роками):

```
SELECT id_cart, status, total_amount, created_date FROM cart_range PARTITION (p_2024) LIMIT 5;
```

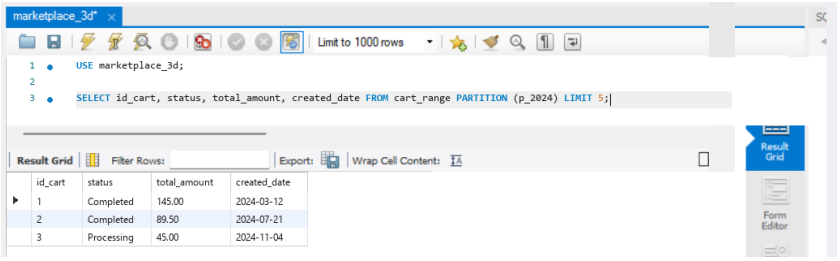


Рис. 5.1. Результат виконання запиту 5.1

Запит 5.2. Розділення таблиці model_3d за допомогою LIST (секціонування моделей за категоріями):

```
SELECT id_model, name, id_category, price FROM model_list PARTITION (p_gamedev) LIMIT 5;
```

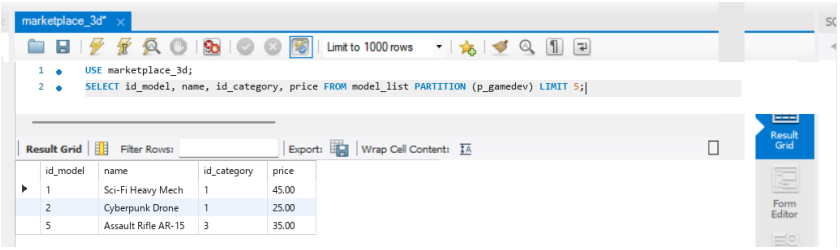


Рис. 5.2. Результат виконання запиту 5.2

Запит 5.3. Розділення таблиці buyer за допомогою HASH (рівномірне хешування покупців за id_buyer):

```
SELECT id_buyer, email, registration_date FROM buyer_hash PARTITION (p0) LIMIT 5;
```

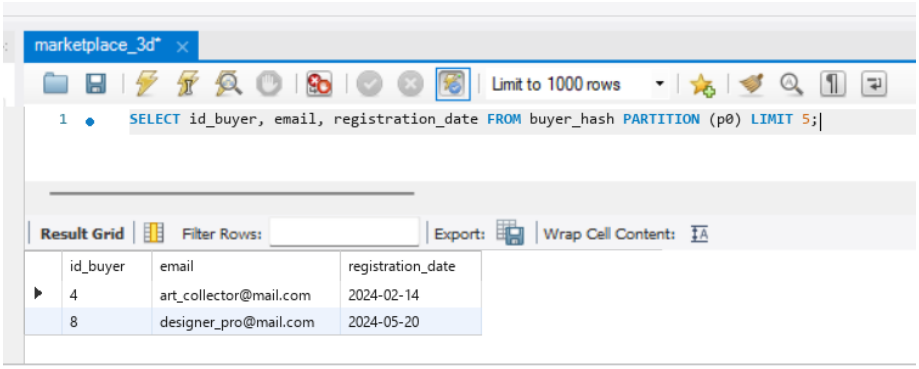


Рис. 5.3. Результат виконання запиту 5.3

Запит 5.4. План виконання запиту з відсіканням секцій (Partition Pruning через EXPLAIN):

```
EXPLAIN PARTITIONS SELECT id_cart, total_amount FROM cart_range WHERE created_date = '2024-05-15';
```

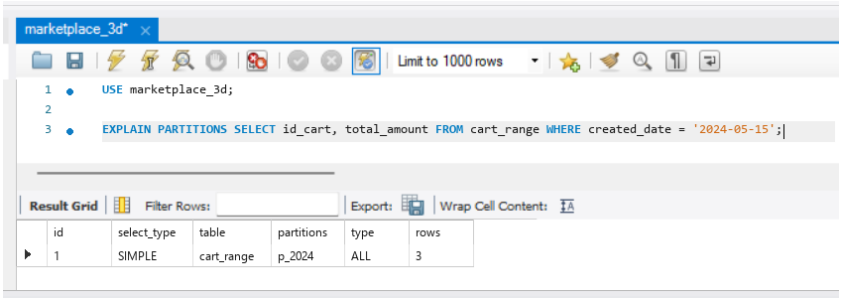


Рис. 5.4. Результат виконання запиту 5.4

Запит 5.5. Перевірка розподілу записів по створених секціях через information_schema.partitions:

```
SELECT table_name, partition_name, table_rows FROM information_schema.partitions WHERE table_name = 'cart_range';
```

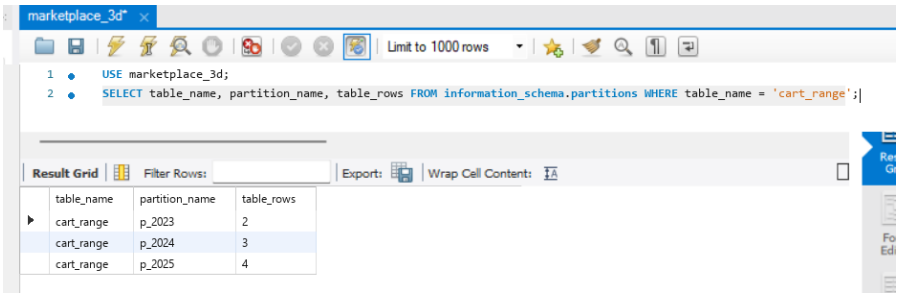


Рис. 5.5. Результат виконання запиту 5.5

Відповіді на контрольні запитання

1. Які предикати можуть використовуватися у виразі WHERE команди SELECT?

Предикати порівняння (=, <>, <, >), перевірки на належність множині IN та NOT IN, квантори ANY/SOME та ALL, перевірки існування рядків EXISTS та NOT EXISTS, а також перевірки на порожнечу IS NULL.

2. Охарактеризуйте роботу кванторів порівняння.

Квантори застосовуються перед множинними підзапитами: IN (рівність будь-якому значенню), NOT IN (нерівність жодному), ANY / SOME (умова істинна, якщо вона виконується хоча б для одного значення зі списку) та ALL (умова повинна виконуватися для всіх значень списку).

3. Як підзапити використовуються у командах UPDATE, INSERT, DELETE?

• INSERT: дозволяє копіювати результати вибірки (INSERT INTO ... SELECT) або динамічно визначати значення ID у виразі VALUES.

• UPDATE: підзапити обчислюють нові значення колонок у SET або фільтрують рядки для модифікації у WHERE.

• DELETE: через WHERE за допомогою підзапиту визначаються застарілі чи зв'язані записи для видалення.

4. Охарактеризуйте роботу оператора CASE.

Це розгалуження в SQL (аналог IF-THEN-ELSE). Дозволяє повертати різні значення за умовами (наприклад, класифікація полігонів Low-Poly / High-Poly) та агрегувати дані за статусами через комбінацію із SUM() або COUNT().

5. Охарактеризуйте роботу різних видів з'єднань.

• INNER JOIN повертає лише рядки з обох таблиць, для яких умова збіглася;

• LEFT JOIN повертає всі рядки з лівої таблиці, а з правої лише збіги, заповнюючи відсутні поля NULL;

• RIGHT JOIN повертає всі рядки з правої таблиці;

• CROSS JOIN формує декартовий добуток (кожен рядок з кожним).

6. Охарактеризуйте операції над множинами UNION, INTERSECT, EXCEPT.

• UNION об'єднує результати двох запитів та видаляє дублікати;

• UNION ALL об'єднує всі записи, зберігаючи дублікати;

• INTERSECT знаходить спільні записи, присутні в обох вибірках (перетин);

• EXCEPT віднімає з першої вибірки всі записи другої (різниця).

7. Запишіть команду декартового добутку таблиць A та B.

SELECT * FROM table_A CROSS JOIN table_B;

Або спрощений запис: SELECT * FROM table_A, table_B;

8. Які типи горизонтального секціонування підтримує MySQL?

MySQL підтримує: RANGE (за діапазонами значень), LIST (за фіксованим списком), HASH (рівномірний розподіл за хешем поля), KEY (хешування системним алгоритмом MySQL) та COLUMNS (розбиття за декількома стовпцями або рядковими типами).

9. Охарактеризуйте тип розбиття таблиці RANGE.

Рядки потрапляють у секцію за числовим діапазоном через VALUES LESS THAN (границя). Останньою часто створюють секцію MAXVALUE для значень, більших за граничні.

10. Охарактеризуйте тип розбиття таблиці LIST.

Рядки розподіляються за точним збігом зі списком дискретних значень через VALUES IN (...). Зручно розбивати дані за категоріями, статусами або кодами міст.

11. Охарактеризуйте тип розбиття таблиці HASH.

Рядки діляться рівномірно між N секціями (PARTITIONS N) за формулою MOD(хеш, N). Використовується для збалансованого навантаження на диск без ручного групування.

Висновки

Під час виконання лабораторної роботи № 5 було вивчено роботу зі складними запитами SELECT та секціонуванням у СУБД MySQL для бази даних marketplace_3d. На практиці написано та протестовано корелюючі й некорелюючі підзапити, оператори внутрішнього та зовнішнього з'єднання (INNER JOIN, LEFT/RIGHT/CROSS JOIN), умовний оператор CASE з агрегацією, операції над множинами (UNION, UNION ALL, INTERSECT, EXCEPT), а також виконано горизонтальне секціонування таблиць за RANGE, LIST та HASH. Усі 25 запитів перевірено у середовищі MySQL Workbench.